

CSCI 6461 | TOPIC 1

Introduction to Computing Systems & Binary Foundations

Abstraction, stored-program organization, data representation, and processor performance

Dr. J. Burns

Computer Systems Architecture — AArch64 Reference Platform

PART 1

Architecture Foundations & Abstraction

What Is Computer Architecture?

- Computer architecture connects **software-visible behavior** with **hardware implementation**.
- Hardware provides an interface that software can rely on.
- The implementation of that interface determines performance and efficiency.
- Architectural design involves correctness, performance, power, area, and cost.

Course Approach

Design approach: RISC principles.

Reference ISA: AArch64.

The 9 Levels of Computer Abstraction

Computers manage immense complexity by enforcing clean hierarchical interfaces:

Application Software	C Programs, Algorithms, Apps
Operating Systems	Device Drivers, Schedulers, Virtual Memory
Architecture (AArch64 ISA)	64-Bit Instructions, X Registers
Organization (Microarchitecture)	Datapaths, Controllers, Pipelines, Caches
Logic Design	Adders, ALUs, Register Files, Decoders
Digital Circuits	and, or, not, xor Logic Gates
Analog Circuits	Differential Amplifiers, Filters, Clocks
Electronic Devices	NMOS / PMOS Field-Effect Transistors
Solid-State Physics	Electron Transport in Silicon Substrates

When Abstractions Leak

Software View

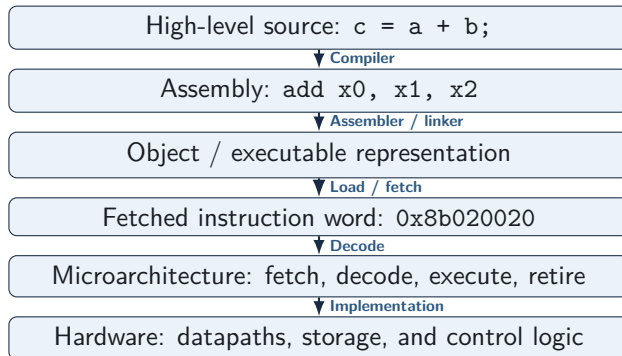
- Variables, Arrays, Branches
- Function calls, Data types

Architectural Effect

- Register allocation, Cache misses
- Branch prediction, Stack frames
- Overflow and sign extension

Performance-sensitive software often exposes lower-level machine behavior.

From Source Code to Execution



The ISA is the boundary between program representation and processor execution.

ISA and Microarchitecture

Instruction Set Architecture

- Instruction encodings & behavior
- Registers x0–x30, sp, PSTATE
- Memory and exception behavior

Microarchitecture

- Pipeline organization, Superscalar width
- In-order or out-of-order execution
- Branch prediction, Cache structures

Different processors can implement the same ISA using different microarchitectures.

RISC Design Principles and AArch64 Encoding

- RISC designs emphasize regular instruction formats and explicit load/store architecture.
- AArch64 instructions are fixed-width **32-bit words**, aligned on 4-byte boundaries.
- Destination (R_d) and source operands (R_n, R_m) use fixed 5-bit register fields ($2^5 = 32$).

Example: 64-Bit Addition (`add x0, x1, x2` $\implies$ `0x8b020020`)

Encoding maps $R_d = x0, R_n = x1, R_m = x2$ within the uniform 32-bit word.

PART 2

Stored-Program Organization & Memory

The Stored-Program Concept & Execution Cycle

- Instructions and data share a unified memory space, encoded as binary numbers.
- The processor maintains a Program Counter (pc) pointing to the next instruction.



Real processors overlap instruction execution via pipelining, while branches and exceptions alter the standard sequential flow.

Memory Topologies: von Neumann, Harvard, Modified Harvard

von Neumann	Pure Harvard	Modified Harvard
Shared memory and bus path; instruction and data access contend for bandwidth.	Separate memories and buses for code and data; enables parallel access.	Unified software address space with split L1 instruction and data caches.

Modern high-performance processors (including AArch64) utilize Modified Harvard topologies.

Modified Harvard & Self-Modifying Code (SMC)

- Split L1 *I*-cache and *D*-cache require explicit synchronization when runtime compilers (JIT) write code as data.
- AArch64 requires explicit cache cleaning, barrier instructions, and instruction invalidation:

Required Synchronization Sequence

```
dc cvau, x0 → dsb ish → ic ivau, x0 → dsb ish → isb
```

Ensures visibility between data modifications and subsequent instruction fetches.

Memory Hierarchy & The Memory Wall

Hierarchy Tiers

- **Registers:** ≈ 1 cycle
- **L1 Cache:** 3–4 cycles
- **L2/L3 Cache:** 10–60 cycles
- **DRAM (Main Memory):** 200+ cycles
- **Persistent Storage:** SSD/NVMe

The Memory Wall

Processor compute capability scales much faster than DRAM latency, making cache management, locality, and prefetching critical to performance.

PART 3

Binary Representation & Arithmetic

Bits and Interpretive Context

- Hardware stores uninterpreted binary bit patterns ($\{0, 1\}^n$).
- An n -bit field yields 2^n combinations; meaning depends entirely on the active instruction context.

Interpretation Perspective	Meaning of 32-bit Word 0x8b020020
AArch64 Machine Instruction	add x0, x1, x2
Unsigned 32-bit Integer	2,332,164,128 ₁₀
Signed 32-bit Integer	−1,962,803,168 ₁₀

Positional Systems & Endianness

- **Positional Binary:** $\text{Value} = \sum_{i=0}^{n-1} b_i 2^i$. Hexadecimal compresses nibbles (4 bits = 1 hex digit).
- **Byte Ordering (Endianness):** Dictates how multi-byte values map to memory addresses.
 - **Little-Endian (AArch64 / x86):** Least Significant Byte (LSB) stored at the lowest address.
 - **Big-Endian:** Most Significant Byte (MSB) stored at the lowest address.

Two's-Complement Representation & Ranges

- Signed integers utilize two's complement, where the MSB carries a negative positional weight (-2^{n-1}).
- **Negation Property:** $-x = \sim x + 1$.

Unsigned Ranges (0 to $2^n - 1$)

- 8-bit: 0 to 255
- 32-bit: 0 to 4.29×10^9
- 64-bit: 0 to 1.84×10^{19}

Two's Complement (-2^{n-1} to $2^{n-1} - 1$)

- 8-bit: -128 to +127
- 32-bit: $\pm 2.14 \times 10^9$
- 64-bit: $\pm 9.22 \times 10^{18}$

Unified Adder Datapath & Extension Rules

- The identical hardware adder supports signed and unsigned addition; subtraction uses $A - B = A + (\sim B) + 1$.
- **Promotion Rules:**
 - **Sign Extension (1drsb):** Replicates the MSB across upper bits to preserve negative values.
 - **Zero Extension (1drb):** Pads upper bits with zeros to preserve unsigned magnitude.

Security Warning

Zero-extending a signed negative integer converts -1 into a massive positive value, breaking array bounds checks.

AArch64 NZCV Condition Flags

Arithmetic instructions can update status flags in PSTATE:

Flag	Name	Evaluation Meaning
N	Negative	Result MSB is 1 (negative signed value)
Z	Zero	All result bits are zero
C	Carry	Unsigned carry-out (add) or No-Borrow ($\geq$, sub)
V	Overflow	Signed arithmetic overflow range violation

Explicit flag-setting instructions use the `s` suffix (adds, subs, or alias `cmp`).

Signed vs. Unsigned Conditional Branching

After a comparison (`cmp x0, x1`), conditional branches test specific flag combinations:

Branch	Context	Flag Condition	Meaning
<code>b.eq</code> / <code>b.ne</code>	Universal	$Z = 1$ / $Z = 0$	Equal / Not Equal
<code>b.lt</code> / <code>b.ge</code>	Signed	$N \neq V$ / $N = V$	Less Than / Greater or Equal
<code>b.lo</code> / <code>b.hs</code>	Unsigned	$C = 0$ / $C = 1$	Lower ($<$) / Higher or Equal ($\geq$)

Choosing the correct mnemonic (`b.lo` for memory pointers vs. `b.lt` for signed integers) is vital for correctness.

Quantitative Processor Performance

Defining Performance and Execution Time

- For a fixed workload, performance is inversely proportional to execution time:

$$\text{Performance} = \frac{1}{\text{Execution Time}}$$

- **CPU Execution Time (T_{CPU}):** Time spent executing instructions on behalf of the application process.
- Clock period and frequency are reciprocals: $T_{\text{clk}} = \frac{1}{f_{\text{clk}}}$.

Latency vs. Throughput

Latency: Time to complete one task.

Throughput: Tasks completed per unit time (e.g., IPC).

Dynamic Instruction Count vs. CPI

- **Dynamic Instruction Count (IC):** The actual number of instructions executed at runtime (dependent on inputs and loops).
- **Cycles Per Instruction (CPI):** Average clock cycles required per instruction:

$$\text{CPI} = \frac{\text{Total Clock Cycles}}{\text{Dynamic Instruction Count}} \quad \text{IPC} = \frac{1}{\text{CPI}}$$

- Modern **superscalar cores** fetch and execute multiple instructions per cycle in parallel ($\text{CPI} < 1.0$, or $\text{IPC} > 1.0$).

The Iron Law of Processor Performance

The Fundamental CPU Performance Equation

$$T_{\text{CPU}} = \text{Instruction Count (IC)} \times \text{CPI} \times T_{\text{clk}} = \frac{\text{IC} \times \text{CPI}}{f_{\text{clk}}}$$

Component	Influenced By	Design Trade-off
IC	Algorithm, Compiler, ISA	CISC vs. RISC instruction density
CPI	Pipeline, Memory hierarchy	Out-of-order complexity, Cache misses
T_{clk}	Pipeline depth, Circuit nodes	Higher frequency requires deeper pipelines

Iron Law: Comparative Evaluation Example

Machine A (Deep Pipeline)

- $IC = 1.0 \times 10^9$
- $CPI = 2.0$
- $f_{clk} = 4.0 \text{ GHz}$

$$T_A = \frac{1.0 \times 10^9 \times 2.0}{4.0 \times 10^9} = \mathbf{0.50 \text{ s}}$$

Machine B (Wide Out-of-Order)

- $IC = 1.2 \times 10^9$ (+20% IC)
- $CPI = 0.8$ (High IPC)
- $f_{clk} = 2.5 \text{ GHz}$

$$T_B = \frac{1.2 \times 10^9 \times 0.8}{2.5 \times 10^9} = \mathbf{0.384 \text{ s}}$$

$$\text{Speedup} = \frac{T_A}{T_B} = \frac{0.50}{0.384} \approx \mathbf{1.30 \times} \text{ faster for Machine B}$$

The Megahertz Myth & Speedup Definition

- **The Megahertz Myth:** Assuming higher clock frequency guarantees better performance. Core efficiency (lower CPI) often outweighs raw clock speed.
- **Speedup (S):** Quantifies performance gains from optimizations:

$$\text{Speedup} = \frac{\text{Execution Time}_{\text{old}}}{\text{Execution Time}_{\text{new}}} = \frac{\text{Performance}_{\text{new}}}{\text{Performance}_{\text{old}}}$$

- If runtime drops from 10 s to 4 s, $\text{Speedup} = \frac{10}{4} = 2.5\times$ (60% runtime reduction).

Amdahl's Law: The Law of Diminishing Returns

Amdahl's Law Equation

$$S_{\text{overall}} = \frac{1}{(1 - f) + \frac{f}{S_{\text{enhanced}}}}$$

- f : Fraction of execution time subjected to acceleration ($0 \leq f \leq 1$).
- S_{enhanced} : Speedup factor achieved exclusively on that fraction.

Worked Example ($f = 0.60, S_{\text{enhanced}} = 10$)

$$S_{\text{overall}} = \frac{1}{(1 - 0.60) + \frac{0.60}{10}} = \frac{1}{0.40 + 0.06} \approx \mathbf{2.17\times}$$

Benchmarking and Architectural Trade-Offs

- **MIPS Flaw:** Instructions per second metrics fail across different ISAs because instruction densities vary. Standard suites (SPEC CPU) measure real application workloads.
- **Key Architectural Trade-Offs:**
 - **Deeper Pipelines:** Shorter clock period vs. higher branch misprediction penalties.
 - **Wider Issue Widths:** Lower CPI vs. exponential hardware complexity and power consumption.
 - **Larger Caches:** Lower miss rates vs. increased die area and latency.

Review and Self-Test Questions

Topic 1 Summary: Architecture & Memory

- **Abstraction Layers:** Systems span from high-level software down to physical solid-state physics, anchored by the ISA contract.
- **ISA vs. Microarchitecture:** The ISA defines software-visible behavior and registers; microarchitecture handles the physical pipeline implementation.
- **Modified Harvard Topologies:** Separates L1 instruction and data caches while presenting a unified software address space.
- **Memory Wall:** Growing latency gaps between fast CPU logic and slow DRAM require deep cache hierarchies and spatial/temporal locality.

Topic 1 Summary: Representation & Performance

- **Two's Complement:** Standard signed integer representation enabling unified adder datapath logic ($A + \sim B + 1$).
- **Condition Flags:** NZCV flags separate unsigned carry (C) from signed overflow (V) behavior.
- **The Iron Law:** CPU execution time depends jointly on instruction count, CPI, and clock period ($T_{\text{CPU}} = \frac{\text{IC} \times \text{CPI}}{f_{\text{clk}}}$).
- **Amdahl's Law:** Overall speedup is strictly constrained by the non-accelerated fraction of code.

Self-Test Diagnostic Questions: Architecture & Binary

- ❶ **ISA vs. Microarchitecture:** Categorize the following:
 - Number of visible registers ($x0$ – $x30$).
 - Size of the L1 instruction cache.
 - Binary machine encoding of the add instruction.
- ❷ **Two's Complement Evaluation:** Convert the 8-bit hex byte $0xa5$ into an unsigned integer and a signed two's complement integer.
- ❸ **Carry vs. Overflow:** In a 4-bit ALU evaluating $1001_2 + 1100_2$, determine the output sum, the Carry flag (C), and the Signed Overflow flag (V).

Self-Test Diagnostic Questions: Performance & Amdahl

- 1 **Iron Law Analysis:** A compiler optimization decreases dynamic instruction count by 20% ($IC_{\text{new}} = 0.80 \times IC_{\text{old}}$), but increases average CPI by 15% ($CPI_{\text{new}} = 1.15 \times CPI_{\text{old}}$). Does performance improve? Calculate the exact speedup.
- 2 **Amdahl's Law Application:** Memory access stalls account for 70% of execution time ($f = 0.70$). If a redesigned memory subsystem accelerates memory accesses by $4\times$ ($S_{\text{enhanced}} = 4$), calculate the overall system speedup.
- 3 **Cache Synchronization:** Why must a dynamic runtime JIT compiler issue explicit cache maintenance operations before executing newly generated machine code on a modified Harvard architecture?

Looking Ahead

Next Lecture Preview

Our Next Topic: RISC vs. CISC: Instruction Set Design Principles & Trade-offs

- **Complexity Distribution:** Examining how different Instruction Set Architectures distribute complexity between software and hardware.
- **Key Trade-offs:** Evaluating instruction encoding, decode complexity, pipeline efficiency, and quantitative performance impact.